The Mathematics of the Murder Plan Solver

State Modeling, Minimax Optimization, and Backward Induction

∅

September 1, 2026

Summary

This document describes the mathematical model and implemented by `murder_plan_solver`. The solver represents a simplified version of the Murder Plan scenario in *Tragedy Looper* as a finite-horizon, two-player zero-sum game. A compact state records relative character positions, relevant intrigue values, and limited card resources. For each state and each number of remaining days, the continuation values of all action pairs form a payoff matrix. Backward dynamic programming evaluates these matrices, while linear programming computes the minimax value and the Mastermind player's optimal mixed strategy. Effect compression, state symmetry, and state and matrix caches make the direct formulation practical. Under the stated model, the Mastermind's average optimal win rate increases from 13.39% in a three-day game to 77.16% in an eight-day game.

1 Introduction

Tragedy Looper[1] is an asymmetric deduction board game in which one player, called the Mastermind, attempts to cause a tragedy while the Protagonists try to prevent it. The solver considers only the Murder Plan scenario and only the rules that directly affect its two associated loss conditions. To make the mathematical roles unambiguous, the Mastermind player is denoted by Alice and the Protagonist player by Bob. The character role called the Brain is distinct from the Mastermind player.

The main question is straightforward: if Alice and Bob both play optimally, what is Alice's probability of completing the murder plan, and how should she assign her cards? A deterministic answer is generally insufficient. If one player always chooses the same action, the opponent can deliberately counter it. Optimal play may therefore require a mixed strategy, meaning a probability distribution over complete card assignments.

Although the board contains only four locations and three relevant characters, the game is not a single matrix game. An action normally changes the state and leaves several days of play. A direct game-tree search also repeats many equivalent subproblems. The solver addresses these issues by combining three ideas. First, the positions of the Brain and Killer are recorded relative to the Key Person, which removes irrelevant absolute locations. Second, backward dynamic programming assigns a continuation value to every reachable state. Third, the one-day decision at each state is solved as a finite zero-sum matrix game using linear programming. The implementation uses several

[1] This is an unofficial fan project and is not affiliated with the game's publisher.

additional reductions, including compression of action pairs with identical effects and caching of repeated payoff matrices.

The method combines standard minimax theory, dynamic programming, and linear programming to solve this concrete finite game. The following sections state the simplified rules and assumptions, define the state transitions, derive the backward minimax calculation, connect the mathematical objects to the Python implementation, and report the resulting win rates and runtime measurements.

2 Game Model and Assumptions

2.1 Board and characters

The board has four locations arranged as a 2×2 grid. A horizontal move, a vertical move, or a diagonal move changes a character's location in the corresponding direction. The model contains three characters:

- the Key Person, denoted by P;
- the Brain, denoted by B; and
- the Killer, denoted by K.

Alice attempts to accumulate intrigue and position the characters so that a Murder Plan condition is satisfied. Bob attempts to prevent this until the last day. Both players know the three character roles from the beginning.

(a) Coordinate system **(b) Relative-state example**

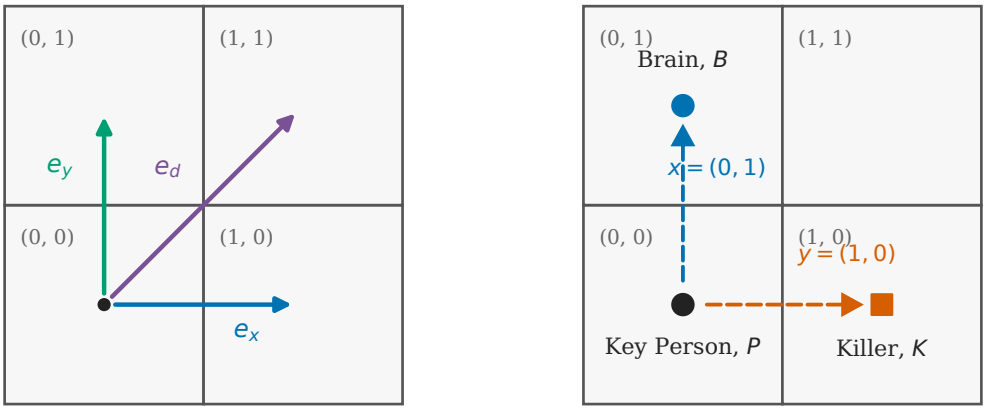

Figure 1: The 2×2 coordinate system and an example of the relative position representation. Absolute board names are unnecessary for the solver because only relative positions affect the modeled rules.

2.2 Cards and turn resolution

Each day, both players assign one card to each of the three characters. A complete action is therefore a triple of card choices rather than a single card. Table 1 summarizes the cards used in the simplified

model. The diagonal move and Intrigue +2 are limited Alice resources, while Bob has three limited Forbid Movement cards. The remaining card types are available again on later days.

Table 1: Cards included in the simplified model.

	Reusable cards		Limited cards	
Player	Card	Copies per day	Card	Initial copies
Alice	Blank	3	Diagonal	1
	Horizontal	1	Intrigue +2	1
	Vertical	1	–	–
	Intrigue +1	1	–	–
Bob	Blank	3	Forbid Movement	3
	Horizontal	3	–	–
	Vertical	3	–	–
	Forbid Intrigue	1	–	–

With all limited cards available, Alice has 136 legal complete actions and Bob has 112, so the initial one-day payoff matrix has size 136×112. These counts decrease as limited cards are spent.

A day is resolved in the following order.

1. Alice and Bob simultaneously choose their complete actions.
2. Intrigue cards are resolved; Forbid Intrigue cancels an intrigue card assigned to the same character.
3. Movement cards are combined and resolved; Forbid Movement prevents movement of its target.
4. Limited cards used during the day are removed from the remaining resources.
5. After the card effects are known, Alice chooses one character in the Brain's location to receive one additional intrigue.
6. The two Murder Plan victory conditions are checked.

Alice wins immediately if the Killer has at least four intrigue, or if the Killer and Key Person share a location while the Key Person has at least two intrigue. If neither condition occurs by the end of day D, Bob wins.

2.3 Scope of the model

The model deliberately excludes most of the full board game. It contains no role deduction, uncertainty about character identities, goodwill, paranoia, additional incidents, or alternative tragedy conditions. It also assumes that both players select their full three-card assignments simultaneously. The results should therefore be interpreted as optimal play within this simplified Murder Plan subgame, not as a solution to every strategic decision in *Tragedy Looper*.

3 State and Transition Model

3.1 Relative state representation

Represent the four locations by

$$\mathbb{F}_2^2 = \{0,1\}^2.$$

The three movement directions are

$$e_x = (1,0), \qquad e_y = (0,1), \qquad e_d = (1,1),$$

where addition is performed in $\mathbb{F}_2^2$. Thus a horizontal or vertical move flips one coordinate and a diagonal move flips both coordinates.

Only relative locations affect the modeled win conditions and the Brain's ability. We therefore place the Key Person at the relative origin. Let $x \in \mathbb{F}_2^2$ be the Brain's location relative to the Key Person and let $y \in \mathbb{F}_2^2$ be the Killer's relative location. Then $x = 0$ means that the Brain meets the Key Person, $y = 0$ means that the Killer meets the Key Person, and $x = y$ means that the Brain meets the Killer. Figure 1 illustrates this representation.

Let $c \in \{0,1,2,3,4\}$ be the Killer's intrigue and $h \in \{0,1,2\}$ the Key Person's intrigue. The values are truncated at their corresponding victory thresholds because larger values cannot change a future decision. Finally, let

$$r = (r_d, r_2, r_f)$$

record whether Alice's diagonal and Intrigue +2 cards remain and how many of Bob's Forbid Movement cards remain. A public game state is

$$S = (x, y, c, h, r).$$

The number of remaining days is written separately as the subscript of the value function.

3.2 Actions and transitions

For a state S, let $\mathcal{A}(S)$ and $\mathcal{B}(S)$ be the legal complete action sets of Alice and Bob. Their pure actions have the form

$$\mathbf{a} = (a_P, a_B, a_K) \in \mathcal{A}(S), \qquad \mathbf{b} = (b_P, b_B, b_K) \in \mathcal{B}(S).$$

For character $i \in \{P, B, K\}$, let u_i and v_i denote the movement directions produced by Alice's and Bob's cards, with zero denoting no movement. When neither card is Forbid Movement, the two movements combine as

$$u \circ v = \begin{cases} u, & u = v, \\ u + v, & u \neq v, \end{cases}$$

where the second line uses addition in $\mathbb{F}_2^2$. A Forbid Movement card instead sets the target's actual movement to zero. Let the resulting movements be m_P, m_B, m_K. Because a move of the Key Person changes both relative coordinates, the updated positions are

$$x' = x + m_B + m_P, \qquad y' = y + m_K + m_P. \tag{1}$$

Let $q_i \in \{0,1,2\}$ be Alice's intrigue-card contribution to character i and let $f_i \in \{0,1\}$ indicate whether Bob forbids that contribution. After card resolution, Alice may use the Brain's ability. We

use $\alpha, \beta \in \{0,1\}$ to indicate an additional intrigue on the Killer or Key Person, respectively. The legal choices are

$$C(S,\mathbf{a},\mathbf{b}) = \left\{ (\alpha,\beta) \in \{0,1\}^2 : \begin{array}{l} \alpha+\beta \le 1, \\ \alpha \le \mathbf{1}_{\{x'=y'\}}, \\ \beta \le \mathbf{1}_{\{x'=0\}} \end{array} \right\}. \tag{2}$$

The new intrigue values are

$$c' = \min\{4, c + (1-f_K)q_K + \alpha\}, \qquad h' = \min\{2, h + (1-f_P)q_P + \beta\}. \tag{3}$$

For a complete action $\mathbf{u}$, let $N_z(\mathbf{u})$ count how many copies of card z it contains. The limited resources then update as

$$\begin{aligned} r_d' &= r_d \mathbf{1}_{\{N_{\text{Diag}}(\mathbf{a})=0\}}, \\ r_2' &= r_2 \mathbf{1}_{\{N_{\text{Int2}}(\mathbf{a})=0\}}, \\ r_f' &= r_f - N_{\text{ForbidMove}}(\mathbf{b}). \end{aligned} \tag{4}$$

In words, a limited Alice card disappears when it is used, and Bob's remaining Forbid Movement count decreases by the number played that day. The full transition is therefore

$$\Phi(S,\mathbf{a},\mathbf{b},\alpha,\beta) = (x',y',c',h',r').$$

3.3 Terminal states and initial positions

The set of states in which Alice has already won is

$$W = \{S : c = 4\} \cup \{S : y = 0 \text{ and } h = 2\}. \tag{5}$$

At the beginning of a game, assume that the absolute locations of the three characters are independent and uniformly distributed. The two relative positions are consequently also independent and uniform:

$$x, y \overset{\text{i.i.d.}}{\sim} \text{Unif}(\mathbb{F}_2^2), \qquad c = h = 0.$$

If r_0 denotes the full initial resources, the average initial value of a D-day game is the mean over the 16 relative position pairs.

4 Minimax Dynamic Programming

4.1 Value function and backward induction

Let $V_d(S)$ be Alice's probability of eventually winning from state S with d days remaining, assuming optimal play by both players. When no days remain,

$$V_0(S) = \mathbf{1}_{\{S \in W\}}. \tag{6}$$

For $d \ge 1$, consider a fixed action pair $(\mathbf{a},\mathbf{b})$. The card effects are resolved before Alice chooses the Brain's target, so she selects the legal target with the highest continuation value:

$$Q_d(S,\mathbf{a},\mathbf{b}) = \max_{(\alpha,\beta)\in C(S,\mathbf{a},\mathbf{b})} V_{d-1}(\Phi(S,\mathbf{a},\mathbf{b},\alpha,\beta)). \tag{7}$$

Index Alice's legal actions by $p = 1,\dots,m$ and Bob's by $q = 1,\dots,n$. Their one-day payoff matrix is

$$M_d(S)_{pq} = Q_d\left(S, \mathbf{a}^{(p)}, \mathbf{b}^{(q)}\right). \tag{8}$$

Unlike an ordinary one-shot payoff, an entry of this matrix is the optimal probability of winning after continuing from the resulting next state.

Let $\Delta(X)$ denote the probability distributions over a finite set X. The state value is the minimax value of the matrix game:

$$V_d(S) = \max_{\sigma \in \Delta(\mathcal{A}(S))} \min_{\tau \in \Delta(\mathcal{B}(S))} \sigma^\top M_d(S) \tau. \tag{9}$$

The finite zero-sum minimax theorem guarantees that the max–min and min–max values are equal [4, 6]. Because every entry of $M_d(S)$ depends only on values with $d-1$ days remaining, the values can be computed in the order

$$V_0 \longrightarrow V_1 \longrightarrow \cdots \longrightarrow V_D.$$

Thus Alice chooses a distribution over complete card assignments that maximizes the win probability she can guarantee against any response by Bob.

4.2 Linear-programming formulation

For a fixed matrix $M = M_d(S)$, Alice chooses a mixed strategy $\sigma = (\sigma_1, \ldots, \sigma_m)$ and a guaranteed value v. It is enough to protect against every pure action of Bob, which gives the linear program

$$\begin{aligned} \max_{\sigma, v} \quad & v \\ \text{subject to} \quad & \sum_{p=1}^{m} \sigma_p M_{pq} \geq v, \quad q = 1, \ldots, n, \\ & \sum_{p=1}^{m} \sigma_p = 1, \\ & \sigma_p \geq 0, \qquad\qquad p = 1, \ldots, m. \end{aligned} \tag{10}$$

The optimal objective is $V_d(S)$ and the optimal vector σ is Alice's mixed card strategy at that state. Bob's strategy can be obtained from the dual minimization problem. The current program needs Alice's strategy for action selection and therefore stores the primal strategy together with the common game value. Standard linear programming methods are described in Boyd and Vandenberghe [1]; the implementation uses SciPy's interface to the HiGHS solvers [3, 5].

Each inequality in Eq. (10) corresponds to one complete action by Bob. Taken together, the inequalities require Alice's mixed strategy to achieve an expected win probability of at least v against every such action.

4.3 Solver outline

Algorithm 1 summarizes the computation for a requested initial state. The forward pass records only states reachable from that request. The backward pass then evaluates each layer after all continuation values needed by that layer are available.

Algorithm 1 Finite-horizon minimax evaluation

Require: Initial state S_0 and horizon D

Forward phase: find relevant states

$L_0 \leftarrow \{S_0\}$
for $t = 0, \ldots, D-2$ **do**
    $L_{t+1} \leftarrow$ all nonterminal successors of states in L_t

Backward phase: solve from the last day to the first

for $t = D-1, D-2, \ldots, 0$ **do**
    $d \leftarrow D - t$
    for all $S \in L_t$ **do**
        for all $(\mathbf{a}, \mathbf{b}) \in \mathcal{A}(S) \times \mathcal{B}(S)$ **do**
            evaluate the successor with Alice's best legal Brain target
            store its value from V_{d-1} in the matching entry of $M_d(S)$
        solve the matrix game in Eq. (10)
        store $V_d(S)$ and Alice's mixed strategy when requested
return $V_D(S_0)$ and Alice's mixed strategy at S_0

The forward phase only determines which states matter for the requested initial position. It starts from S_0, applies every legal action pair, and collects the nonwinning successors one day at a time. A successor in which Alice has already won does not need to be expanded further because its value is already one.

The backward phase provides the actual solution. With one day remaining, every action pair is easy to evaluate: its payoff is one if Alice can reach a winning state after choosing the Brain's target, and zero otherwise. Solving this first matrix gives V_1. With two days remaining, the entries are no longer just zeros and ones; they are the already computed one-day values of the successor states. Repeating this process eventually produces $V_D(S_0)$. This is why the algorithm must work backward even though the relevant states are first found forward.

Caching and symmetry do not change this logic. They only avoid solving a state or payoff matrix again when the same subproblem appears through another game history.

The final policy is state-dependent. During play, Alice samples a complete card assignment from the stored mixed strategy. After both card assignments are revealed and resolved, she compares the legal Brain targets using the next-day values and chooses a maximizing target. Choosing only the action with the largest probability would destroy the protection provided by the mixed strategy; the probabilities must be used as an actual random distribution.

5 Solver Architecture

The solver is implemented in Python. The module `model.py` defines states, actions, strategy entries, and returned results. The module `rules.py` generates legal actions and evaluates one-day card effects. The module `solver.py` builds reachable layers, constructs payoff matrices, performs backward evaluation, and calls `scipy.optimize.linprog`. A separate command-line interface converts user input to solver states. The complete source is available in the project repository [2].

Table 2: Correspondence between the mathematical model and the main source modules.

Mathematical object or operation	Implementation module
States, actions, strategies, and returned values	`model.py`
Legal actions and the transition Φ	`rules.py`
Reachable layers, payoff matrices, backward evaluation, and LP solves	`solver.py`
User input and strategy lookup	command-line interface

5.1 Effect compression

The Cartesian product $\mathcal{A}(S) \times \mathcal{B}(S)$ contains many action pairs that produce exactly the same movement, intrigue increments, and remaining resources. The implementation represents each distinct result by an internal `Effect`. Continuation values are computed once per distinct effect and then expanded back to the full payoff matrix through an index array. This preserves the original pure actions, which are needed for the mixed strategy, while avoiding repeated state transitions inside a matrix.

Legal actions and effect tables depend on the remaining limited cards but not on character positions or intrigue. They are therefore cached by inventory. This is separate from effect compression: compression removes duplicate work inside one matrix, while the inventory cache reuses action data across many states.

5.2 State and matrix caches

If two histories reach the same pair (S, d), their future games are identical. State values are stored in memory and reused whenever this occurs. The map is also symmetric under exchanging the horizontal and vertical axes, provided that the corresponding movement labels are exchanged. The value cache uses a canonical representative of these two symmetric states. Only the value is shared directly because a returned strategy contains direction-specific card names.

Different states may also generate exactly the same numerical payoff matrix. The solver hashes the matrix shape and entries with BLAKE2b and caches the linear-programming result. In an instrumented three-day average-position run, 3,571 matrix solutions were requested, but only 2,862 matrix contents were distinct; 709 requests, or 19.85%, reused an existing solution. With state symmetry and matrix caching enabled together, 2,231 canonical state values required 1,832 actual LP calls. The cache occupied approximately 1.12 MiB.

Caching every state-transition call was tested but not retained. Although many calls repeat, storing hundreds of thousands of tuples of successor states uses substantial memory. In this case, recomputing the inexpensive transition was a better time–space tradeoff.

6 Computational Results

All reported values were generated locally from the same implementation. The solver enumerates actions and states rather than sampling game trajectories, so the win-rate results are deterministic up to floating-point tolerances in the LP solver. Runtime measurements should be interpreted as within-machine comparisons: the important quantity is the difference between configurations, not a universal absolute running time.

6.1 Optimal win rates

We evaluate game lengths from three to eight days. Table 3 compares two fixed initial configurations with the average over all 16 pairs of relative positions. All tests start with zero intrigue and full limited-card resources.

Table 3: Alice's optimal win rate under three initial-position settings.

D	$x = (0,1),\ y = (1,0)$	$x = y = (0,0)$	Uniform average
3	14.51%	13.43%	13.39%
4	26.74%	25.91%	26.01%
5	41.34%	39.36%	40.60%
6	55.42%	53.53%	55.13%
7	67.67%	66.08%	67.43%
8	77.37%	76.08%	77.16%

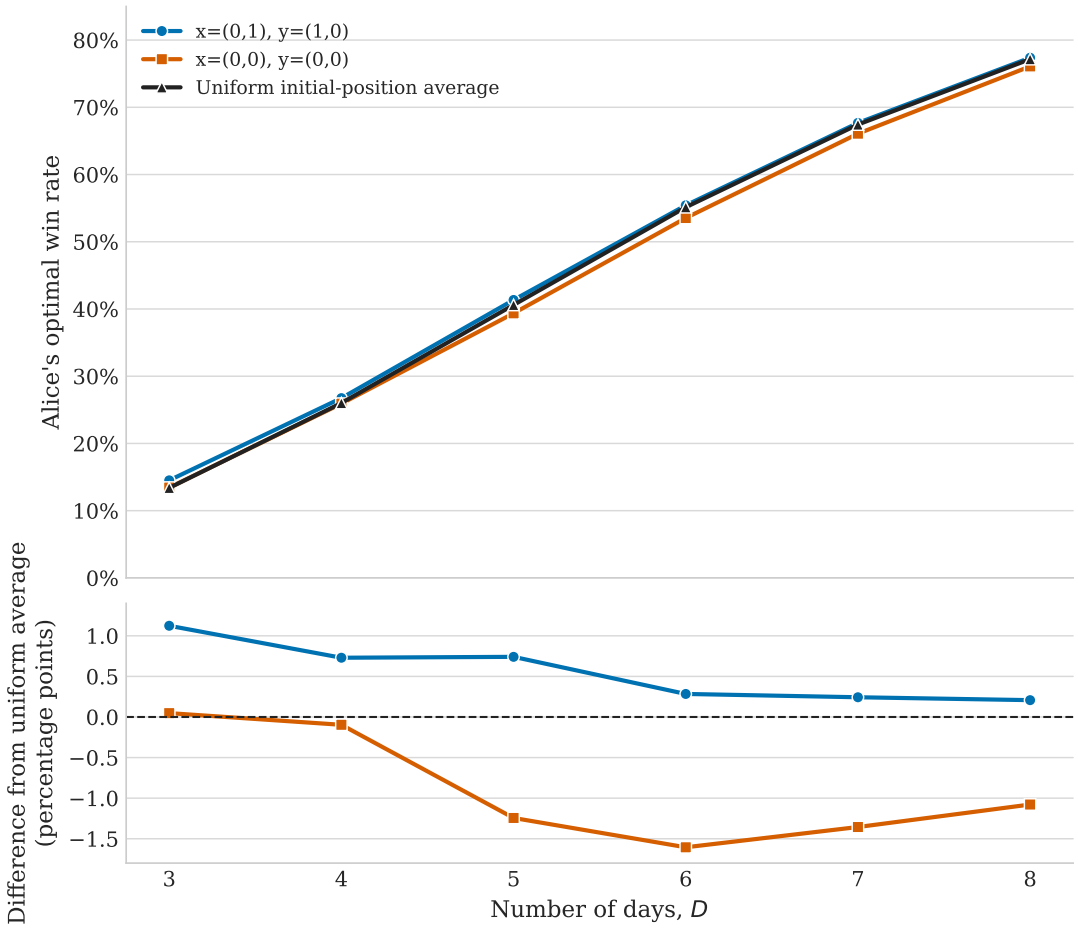

Figure 2: Optimal win rates for two fixed initial states and the uniform average. The lower panel shows percentage-point differences from the uniform initial-position average.

As expected, the number of available days is the dominant factor in these experiments. The

average optimal win rate rises from 13.39% at $D = 3$ to 77.16% at $D = 8$, as shown in Figure 2. The two displayed fixed states remain close to the average, with differences of only a few percentage points. This limited sample suggests that the initial relative positions may matter less than the horizon, although a statement about all positions would require reporting the full range over the 16 configurations.

One mildly counterintuitive observation is that placing all three characters together does not improve Alice's value in these tests. Within the simplified model, Bob can focus entirely on disrupting the Murder Plan and has enough movement cards to separate the characters. In a complete game, Bob may need to spend cards defending against other tragedy conditions, so this observation should not be generalized beyond the present assumptions.

6.2 Linear-programming presolve

After caching, much of the remaining runtime comes from repeatedly starting small LP solves. HiGHS normally performs a presolve phase that removes redundant constraints and reduces a problem before optimization. Presolve is valuable for many large problems, but its fixed preparation cost can dominate the small, similarly structured matrices in this solver.

Disabling presolve changed computed state values by at most 1.665×10^{-15} in the comparison, while reducing runtime for every tested horizon. Figure 3 shows that the reduction grows from 22.6% at $D = 3$ to 39.1% at $D = 8$. At eight days, total measured runtime fell from 64.16 seconds to 39.07 seconds. This is an implementation-specific result, not a claim that presolve should generally be disabled: the outcome follows from solving a large number of small LPs rather than a few large ones.

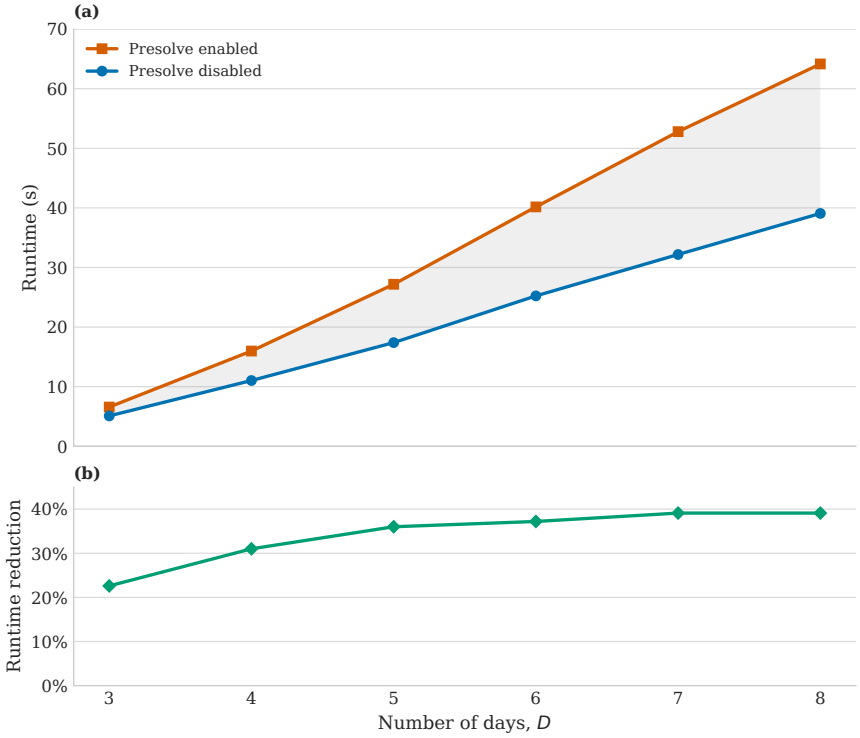

Figure 3: Runtime with HiGHS presolve enabled and disabled. The lower panel reports the relative runtime reduction obtained by disabling presolve.

Thread-level parallelism was also tested for the independent LP calls, but the observed improvement was modest: the three-day run decreased from 4.59 seconds with one thread to 4.00 seconds with eight threads. The final implementation therefore keeps the simpler single-threaded design.

7 Scope and Limitations

The largest limitation is the model scope. Real games contain information and objectives not represented by S, and those factors can change both players' card allocations. The results describe the isolated Murder Plan mechanism under full knowledge of the three relevant roles. They should not be treated as a complete strategy for every loop or script.

The computation is exact in the sense that legal states and actions are enumerated rather than estimated by simulation. The linear programs, however, are solved with floating-point arithmetic, so values and strategy probabilities are numerical approximations within solver tolerances. Finally, adding more characters, resources, or information states would enlarge both the state space and the one-day action matrices. The current reductions help, but they do not remove this general growth.

8 Conclusion

The solver formulates a simplified Murder Plan scenario as a finite-horizon zero-sum game. Relative coordinates reduce the location representation, and the remaining intrigue and limited-card information form a compact public state. Backward dynamic programming converts future state values into a one-day payoff matrix at each state, while linear programming produces the minimax value and Alice's optimal mixed action.

The implementation shows how several small engineering choices can make this direct formulation practical. Compressing identical card effects avoids repeated transition work, state and matrix caches reuse equivalent subproblems, and disabling presolve reduces overhead for the many small linear programs. Under the simplified rules, Alice's average optimal win rate grows substantially with the number of days, reaching 77.16% for an eight-day game. The solver therefore serves both as an analysis of the scenario and as a concrete example of applying minimax optimization to a sequential game.

References

[1] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. URL `https://web.stanford.edu/~boyd/cvxbook/`.

[2] emptysetvvvv. Murder Plan Solver. GitHub repository, 2026. URL `https://github.com/emptysetvvvv/murder-plan-solver`. Accessed August 31, 2026.

[3] Qi Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10:119–142, 2018. doi: 10.1007/s12532-017-0130-5.

[4] Roger B. Myerson. *Game Theory: Analysis of Conflict*. Harvard University Press, 1991.

[5] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.

[6] John von Neumann. Zur Theorie der Gesellschaftsspiele. *Mathematische Annalen*, 100:295–320, 1928. doi: 10.1007/BF01448847.